

 Politechnika Wrocławska	Projektowanie efektywnych algorytmów – projekt - Zadanie 3	
	Maksymilian Tara 264000	Data oddania sprawozdania: 15.01.2024.
	Poniedziałek 15:15 – 16:55	Dr inż. Jarosław Mierzwa

Spis Treści

1. Wstęp.....	2
2. Omówienie Algorytmu.....	2
2.1. Opis Działania Algorytmu	2
2.2. Opis Implementacji Algorytmu	3
2.2.1. Populacja Startowa.....	4
2.2.2. Ocena Przystosowania.....	5
2.2.3. Krzyżowanie	5
2.2.4. Mutacja.....	7
2.2.5. Elityzm	10
2.3. Opis Klas w Programie.....	11
2.3.1. Klasa main	11
2.3.2. Klasa matrix	11
2.3.3. Klasa specimen.....	11
2.3.4. Klasa geneticAlgorithm	11
3. Plan Eksperymentu	11
4. Wyniki Testów	13
4.1. Graf mały – ftv47.atsp.....	13
4.2. Graf średni – ftv170.atsp.....	14
4.3. Graf duży – rbg403.atsp	15
5. Wnioski	16
5.1. Porównanie z Algorytmem Symulowanego Wyżarzania.....	17

1. Wstęp

Rozpatrywanym problemem, jest problem komiwojażera (ang. TSP – Travelling Salesman Problem). Polega on, na znalezieniu minimalnego cyklu Hamiltona w pełnym grafie ważony. Cykl Hamiltona to taki cykl, w którym w grafie każdy wierzchołek odwiedzany jest dokładnie raz, a następnie następuje powrót do wierzchołka startowego. Minimalny – oznacza, że wagi przypisane do krawędzi grafu (połączeń między wierzchołkami) przez które przechodzi nasz znaleziony cykl, mają dawać najmniejszą możliwą sumę. Problem ten można przedstawić na przykładzie z życia jako podróżującego sprzedawcę, który musi odwiedzić każde z n miast na swojej określonej drodze (zaczyna z punktu A i na końcu chce znaleźć się w punkcie B) jak najniższym kosztem.

W tym eksperymencie problem rozpatrywany będzie dla grafów asymetrycznych – waga ścieżki (długość lub koszt drogi) z wierzchołka A do wierzchołka B, może być inna niż z wierzchołka B do wierzchołka A (w drugą stronę).

Problem TSP jest problemem NP-trudnym.

W celu rozwiązania tego problemu został zaimplementowany algorytm Genetyczny.

2. Omówienie Algorytmu

Algorytm Genetyczny to algorytm heurystyczny, podobnie jak rozważany w poprzednim zadaniu algorytm Symulowanego Wyżarzania. W poprzednim eksperymencie zostały już omówione zalety i wady algorytmów heurystycznych w porównaniu do „tradycyjnych” deterministycznych algorytmów. W tym przypadku porównane ze sobą zostaną 2 algorytmy heurystyczne – Genetyczny i Symulowanego Wyżarzania.

Algorytm Genetyczny zalicza się do grupy algorytmów ewolucyjnych. Wzoruje się on na biologicznym zjawisku ewolucji - istnieje pewna populacja osobników, w której każdy z nich ma swoje geny – w tym przypadku są to miasta odwiedzane w ścieżce. W celu znalezienia najlepszego rozwiązania, osobniki z populacji będą się rozmnażać i co za tym idzie, mieszać swoje geny tworząc następne pokolenie.

2.1. Opis Działania Algorytmu

Teoria działania algorytmu w kolejnych krokach:

1. Losowanie Populacji Startowej.
2. Ocena przystosowania – w tym przypadku obliczenie długości trasy dla każdego osobnika. Selekcja na podstawie przystosowania.
3. Krzyżowanie – przemieszanie genów (ścieżek) z 2 osobników w celu stworzenia dziecka
4. Mutacja – wprowadzenie drobnych losowych zmian w genach (ścieżce) dziecka
5. Po stworzeniu całego następnego pokolenia przez wykonywanie kroków 3 i 4, powrót do kroku 2, lub zakończenie działania algorytmu, jeśli upłynął założony czas jego działania.

2.2. Opis Implementacji Algorytmu

```
startingPopulation();
std::chrono::time_point breakPoint = std::chrono::system_clock::now() +
std::chrono::duration_cast<std::chrono::seconds>(std::chrono::duration<int>
(maxTime));
std::random_device rd;
std::mt19937 generate(rd());
std::uniform_real_distribution<double> dist(0.0, 1.0);

auto startTime = std::chrono::system_clock::now();

do{
    fitness();
    //Zapisanie najlepszego rozwiazania
    if(currentPopulation[currentPopulation.size()-1].cost < bestDistance){
        bestSolution = currentPopulation[currentPopulation.size()-1].path;
        bestDistance = currentPopulation[currentPopulation.size()-1].cost;
        bestSolutionFoundTime =
std::chrono::duration_cast<std::chrono::microseconds>(std::chrono::system_c
lock::now() - startTime).count();
        rozwiazania.push_back(bestDistance);
        czasy.push_back(bestSolutionFoundTime);
    }
    //Selection + cross
    for (int i = 0; i < populationSize - 1; i++) {
        if(crossFactor > dist(generate)) {
            cross();
        }
        else{
            nextPopulation.push_back(currentPopulation[i]);
        }
    }
    //Mutation
    if(mutationChoice == 1) {
        for (int i = 0; i < nextPopulation.size(); i++) {
            if (mutateFactor > dist(generate)) {
                mutatel(&nextPopulation[i]);
            }
        }
    }
    else if(mutationChoice == 2){
        for (int i = 0; i < nextPopulation.size(); i++) {
            if (mutateFactor > dist(generate)) {
                mutate2(&nextPopulation[i]);
            }
        }
    }
    //Elitism
    nextPopulation.push_back(currentPopulation[currentPopulation.size()-
1]);
    //Zapisanie nowej populacji jako aktualnej
    currentPopulation = std::move(nextPopulation);
    nextPopulation.clear();
    //nextPopulation.shrink_to_fit();
    parents.clear();
    //parents.shrink_to_fit();

}while ((breakPoint - std::chrono::system_clock::now()).count() > 0);
```

Powyżej widoczny jest kod algorytmu. Jego wykonanie zaczyna się od stworzenia populacji startowej o zadanym rozmiarze:

2.2.1. Populacja Startowa

```
void geneticAlgorithm::startingPopulation() {
    for (int i = 0; i < populationSize; i++){
        std::vector<int> sciezka(numberOfCities);
        //wypelnienie wektora wszystkimi wierzchołkami
        std::iota(sciezka.begin(), sciezka.end(), 0);
        //random shuffle
        std::random_device rd;
        std::mt19937 generate(rd());
        std::shuffle(sciezka.begin(), sciezka.end(), generate);
        //dodanie wierzchołka startowego na koniec (cykl hamiltona)
        sciezka.push_back(sciezka[0]);
        //obliczenie kosztu
        int koszt = 0;
        for (int j = 0; j < sciezka.size() - 1; j++){
            koszt += distanceMatrix[sciezka[j]][sciezka[j+1]];
        }
        //dodanie do populacji
        currentPopulation.emplace_back(sciezka, koszt);

        if(koszt < bestDistance){
            bestSolution = sciezka;
            bestDistance = koszt;
        }
    }
    rozwiazania.push_back(bestDistance);
    czasy.push_back(bestSolutionFoundTime);
    displayResults();
}
```

W celu stworzenia populacji początkowej, wektor wypełniany jest liczbami od 0 do liczby miast w grafie. Następnie te liczby są losowo mieszane i na koniec dokładane jest miasto z pozycji 0, czyli startowej. Następnie obliczany jest koszt tej ścieżki, tworzony jest osobnik, który łąduje w wektorze zawierającym aktualną populację.

Następnie mamy funkcję przystosowania:

2.2.2. Ocena Przystosowania

```
void geneticAlgorithm::fitness() {
    //sortowanie populacji po koszcie malejaco (najgorsze na poczatku)
    std::sort(currentPopulation.begin(), currentPopulation.end(), [](const
specimen& lhs, const specimen& rhs){return lhs.cost > rhs.cost;});

    //dodanie osobnikow do wektora selekcji rodzicow - ilosc zalezna od
rangi
    for (int i = 0; i < currentPopulation.size(); i++){
        for (int j = 0; j < i+1; j++) {
            parents.push_back(currentPopulation[i]);
        }
    }
    //losowe przetasowanie tablicy rodzicow
    std::random_device rd;
    std::mt19937 generate(rd());
    std::shuffle(parents.begin(), parents.end(), generate);
}
```

Aktualna populacja sortowana jest malejąco względem kosztów – najmniejsze na końcu. Następnie do tymczasowego wektora rodziców, z którego będą wybierane osobniki do krzyżowania dodawana odpowiednia ilość kopii osobnika, zależna od jego pozycji w posortowanej populacji. Na koniec lista ta jest losowo mieszana. Powoduje to większą szansę na selekcję najlepszych osobników w procesie krzyżowania – selekcja rankingowa.

Potem przechodzimy do krzyżowania:

2.2.3. Krzyżowanie

```
void geneticAlgorithm::cross() {
    std::random_device rng;
    std::mt19937 generate(rng());
    std::uniform_int_distribution<> distribution(0, parents.size() - 2);

    int r1 = distribution(generate);
    int r2 = distribution(generate);

    //kopie wektorow
    std::vector rodzic1 = parents[r1].path;
    std::vector rodzic2 = parents[r2].path;
    rodzic1.pop_back();
    rodzic2.pop_back();

    std::vector<int> dziecko(numberOfCities, INT_MAX);

    //wybor przedzialu z rodzic1
    std::uniform_int_distribution<> distribution2(0, rodzic1.size() - 2);
    int start = distribution2(generate);
    int end;
    do {
        end = distribution2(generate);
    }while (end == start);
    if (start > end) {
        std::swap(start, end);
    }
}
```

```

//skopiowanie wybranego przedziału z rodzic1 do dziecka
//dziecko.assign(rodzic1.begin() + start, rodzic1.begin() + end);
for (int i = start; i <= end; i++){
    dziecko[i] = rodzic1[i];
}

//wypełnienie reszty dziecka w kolejności unikalnych wierzchołków z
rodzic2
for(int i = 0; i < rodzic2.size(); i++){
    int gen = rodzic2[i];
    bool missing = true;
    for (int j = 0; j < dziecko.size(); j++){
        if (dziecko[j] == gen){
            missing = false;
            break;
        }
    }
    if(missing){
        for (int j = 0; j < dziecko.size(); j++){
            if(dziecko[j] == INT_MAX){
                dziecko[j] = gen;
                break;
            }
        }
    }
}

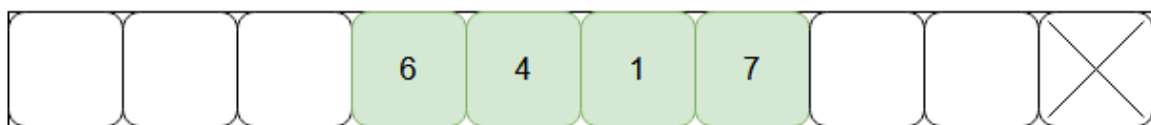
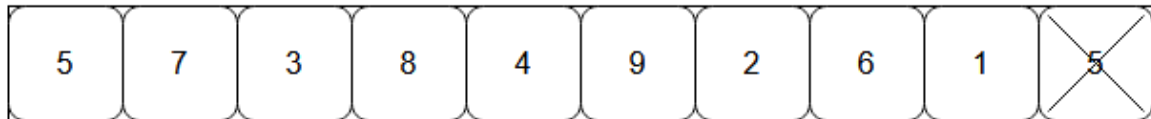
dziecko.push_back(dziecko[0]);
int koszt = 0;
for (int j = 0; j < dziecko.size() - 1; j++){
    koszt += distanceMatrix[dziecko[j]][dziecko[j+1]];
}
//dodanie do nowej populacji
nextPopulation.emplace_back(dziecko, koszt);
}

```

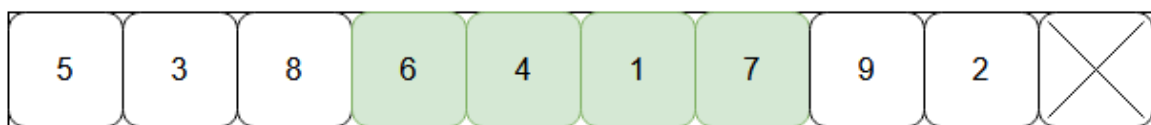
Jest to Order Crossover. Krzyżowanie to polega na:

- Wylosowaniu 2 rodziców z tymczasowego wektora rodziców – selekcja rankingowa.
- Wylosowaniu przedziału ze ścieżki 1 rodzica i przypisaniu go do dziecka na tych samych pozycjach.
- Uzupelnieniu pozostałych pozycji w ścieżce dziecka brakującymi wierzchołkami, w kolejności w jakiej występują w 2 rodzicu.

Szansa na krzyżowanie wynosi 80%, jeżeli osobnik nie skrzyżuje się z żadnym, przechodzi po prostu do następnego pokolenia.



Brakujące: 2, 3, 5, 8, 9 - uzupełniane w kolejności występowania w rodzicu 2



Następnym krokiem jest mutacja:

2.2.4. Mutacja

Zostały zaimplementowane 2 metody mutacji:

- 1 – Zamiana 2 losowych wierzchołków
- 2 – Odwrócenie kawałka ścieżki, znajdującego się pomiędzy 2 losowymi wierzchołkami

Szansa na mutację wynosi 1%. Jeżeli osobnik nie zmutuje, nie dzieje się nic innego.

2.2.4.1. Mutacja 1 – zamiana 2 losowych wierzchołków

```
void geneticAlgorithm::mutate1(specimen* osobnik) {
    std::random_device rng;
    std::mt19937 gen(rng());
    std::uniform_int_distribution<> distribution(0, osobnik->path.size() -
2);

    int swap1 = distribution(gen);
    int swap2 = distribution(gen);
    //Zabezpieczenie zeby nie wylosowaly sie 2 te same
    while(swap2 == swap1){
        swap2 = distribution(gen);
    }
}
```

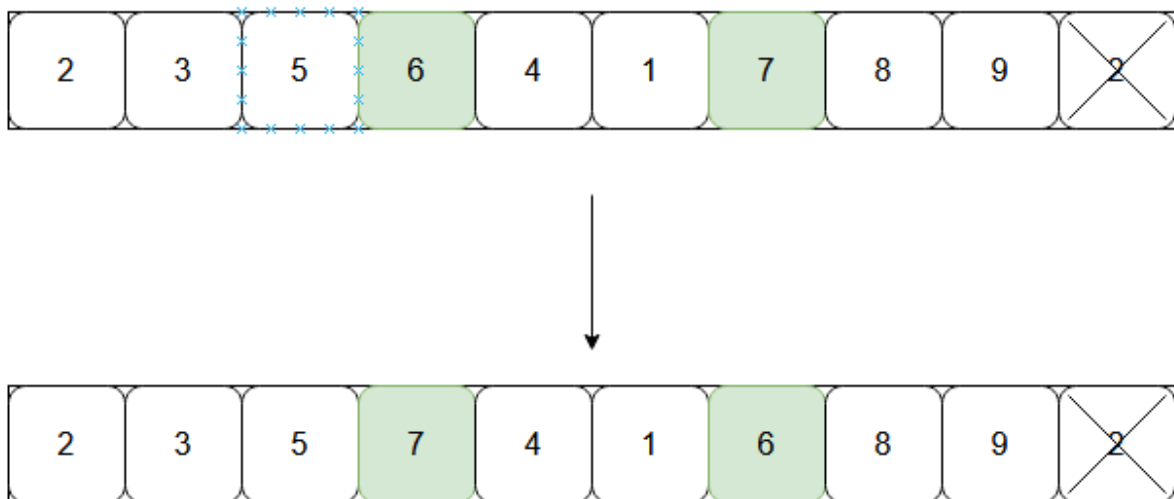
```

osobnik->path.pop_back();
//zamiana 2 losowych wierzchołkow
std::vector<int> tempOsobnik = osobnik->path;
osobnik->path[swap1] = tempOsobnik[swap2];
osobnik->path[swap2] = tempOsobnik[swap1];
//
osobnik->path.push_back(osobnik->path[0]);

//ponowne obliczenie kosztu sciezki
int koszt = 0;
for (int j = 0; j < osobnik->path.size() - 1; j++){
    koszt += distanceMatrix[osobnik->path[j]][osobnik->path[j+1]];
}
osobnik->cost = koszt;
}

```

Losowane są 2 wierzchołki ze stworzonego poprzednio dziecka (bądź osobnika, który przeszedł do następnego pokolenia bez krzyżowania) i zamieniane miejscami.



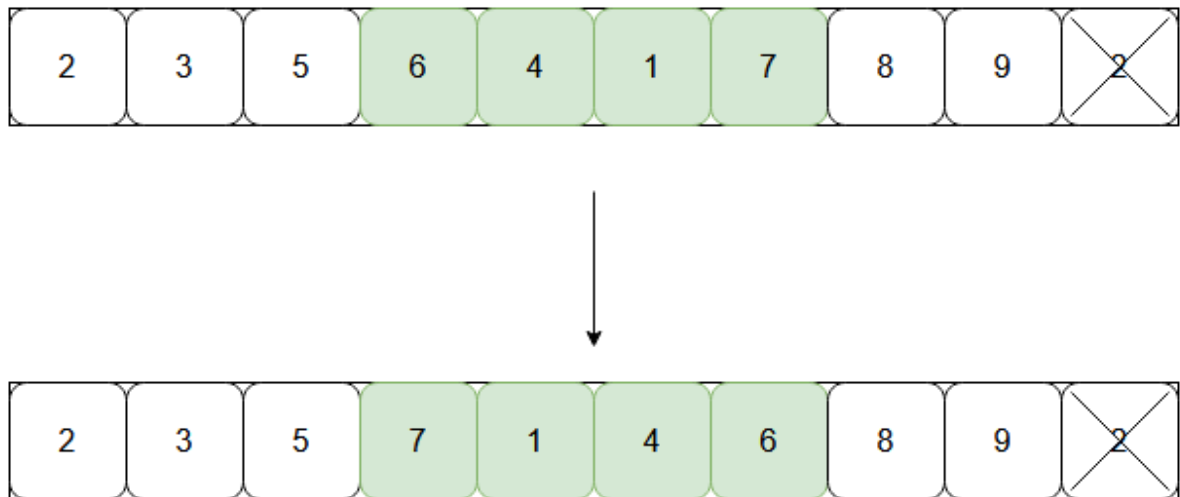
2.2.4.2. Mutacja 2 – Odwrócenie fragmentu ścieżki

```
void geneticAlgorithm::mutate2(specimen* osobnik) {
    std::random_device rng;
    std::mt19937 gen(rng());
    std::uniform_int_distribution<> distribution(0, osobnik->path.size() -
2);

    int start = distribution(gen);
    int end = distribution(gen);
    //Zabezpieczenie zeby nie wylosowaly sie 2 te same
    while(end == start){
        end = distribution(gen);
    }
    if (start > end) {
        std::swap(start, end);
    }
    //
    osobnik->path.pop_back();
    //
    std::vector<int> tempSegment;
    tempSegment.assign(osobnik->path.begin()+start, osobnik-
>path.begin()+end);
    int i = tempSegment.size() - 1;
    do{
        osobnik->path[start] = tempSegment[i];
        start++;
        i--;
    }while(start < end);
    //
    osobnik->path.push_back(osobnik->path[0]);

    //ponowne obliczenie kosztu sciezki
    int koszt = 0;
    for (int j = 0; j < osobnik->path.size() - 1; j++){
        koszt += distanceMatrix[osobnik->path[j]][osobnik->path[j+1]];
    }
    osobnik->cost = koszt;
}
```

Polega ona na wylosowaniu 2 wierzchołków, podobnie jak w poprzedniej metodzie mutacji. Jednak tym razem zapisujemy cały fragment ścieżki pomiędzy tymi 2 wierzchołkami do tymczasowego wektora i zapisujemy go na odpowiednie miejsca z powrotem do tego samego osobnika w odwrotnej kolejności.



2.2.5. Elityzm

```
//Elitism  
nextPopulation.push_back(currentPopulation[currentPopulation.size()-1]);
```

Zabieg ten polega na zachowywaniu najlepszego osobnika z danej populacji, bez żadnych zmian. Osobnik o najniższym koszcie ścieżki w danym pokoleniu zawsze przechodzi do następnego.

Następnie wykonywana jest pętla od Oceny Przystosowania na następnej populacji.

2.3. Opis Klas w Programie

2.3.1. Klasa main

W tej klasie znajduje się menu, z którego sterujemy całym programem i wywoływane są poszczególne funkcje programu, takie jak wczytanie grafu, podanie parametrów startowych i uruchomienie algorytmu.

2.3.2. Klasa matrix

W tej klasie znajdują się funkcje umożliwiające wczytanie grafu z pliku o rozszerzeniu .atsp do macierzy sąsiedztwa oraz ilości jego wierzchołków.

2.3.3. Klasa specimen

Klasa ta zawiera definicję obiektu osobnika, który zawiera:

- Wektor – ścieżka
- Koszt – całkowity koszt ścieżki, potrzebny do oceny przystosowania

2.3.4. Klasa geneticAlgorithm

Ta Klasa zawiera wszystkie zmienne, struktury danych oraz funkcje potrzebne do działania algorytmu, jak również wyświetlania danych po jego zakończeniu czy zapisu wyników do pliku.

3. Plan Eksperymentu

Grafy, dla których próbowano znaleźć jak najlepsze rozwiązanie zapisane są w plikach:

- ftv47.atsp
- ftv170.atsp
- rbg403.atsp

Proponowane czasy działania algorytmów to kolejno:

- 2 minuty (120 sekund) dla grafu małego – ftv47
- 4 minuty (240 sekund) dla grafu średniego – ftv170
- 6 minut (360 sekund) dla grafu dużego – rbg403

Parametry:

- Współczynnik krzyżowania = 0.8
- Współczynnik mutacji = 0.01
- Rozmiar populacji: 100, 200, 400 - dla każdego grafu.

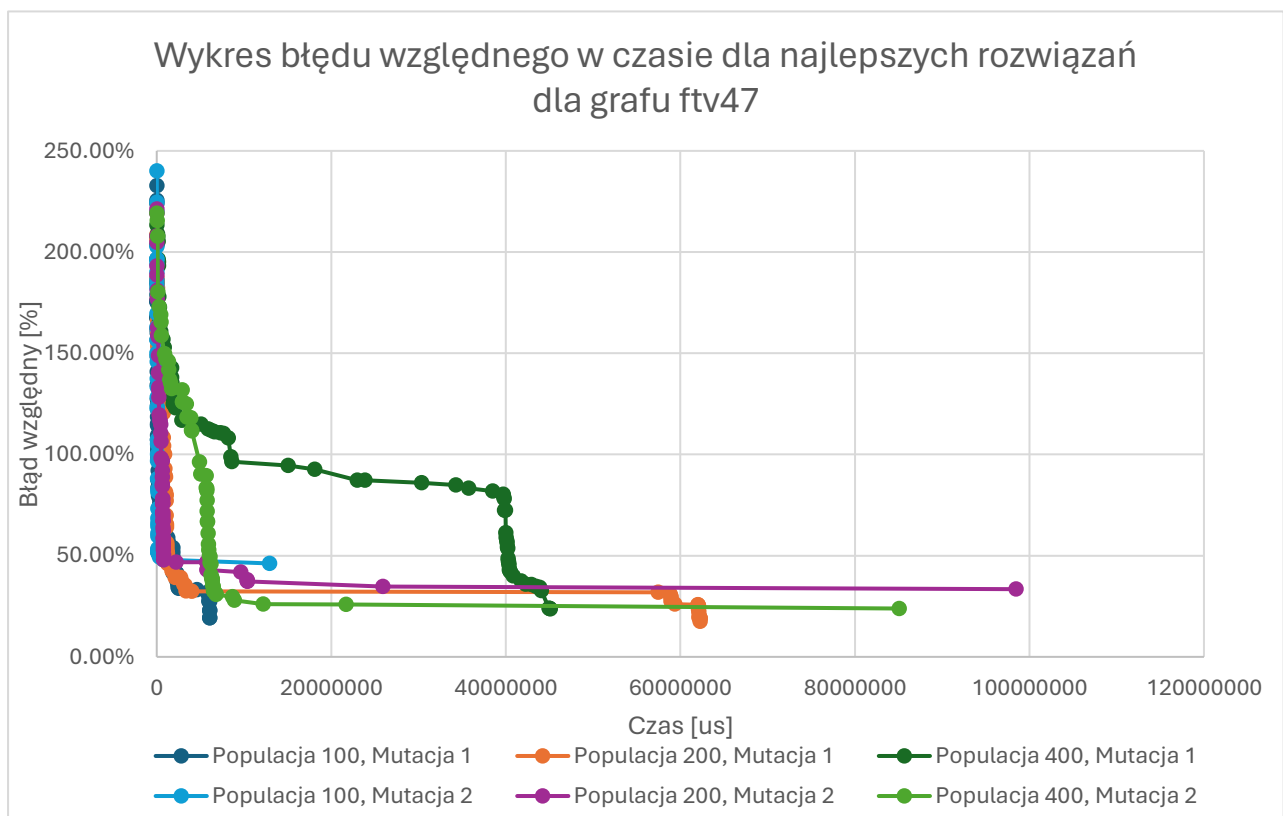
Dla każdego grafu i rozmiaru populacji zostały przetestowane obydwie zaimplementowane metody mutacji. Dla każdej kombinacji zostało wykonane 5 testów – przykładowo dla grafu ftv47 dla populacji = 100 wykonano 5 testów z metodą mutacji 1 oraz 5 z metodą mutacji 2, co daje nam łącznie 10 testów dla każdej wielkości populacji.

Na podstawie wyników testów, obliczone zostały błędy względne w stosunku do optymalnej ścieżki dla tego grafu, a same wyniki przedstawione zostały w tabelach.

4. Wyniki Testów

4.1. Graf mały – ftv47.atsp

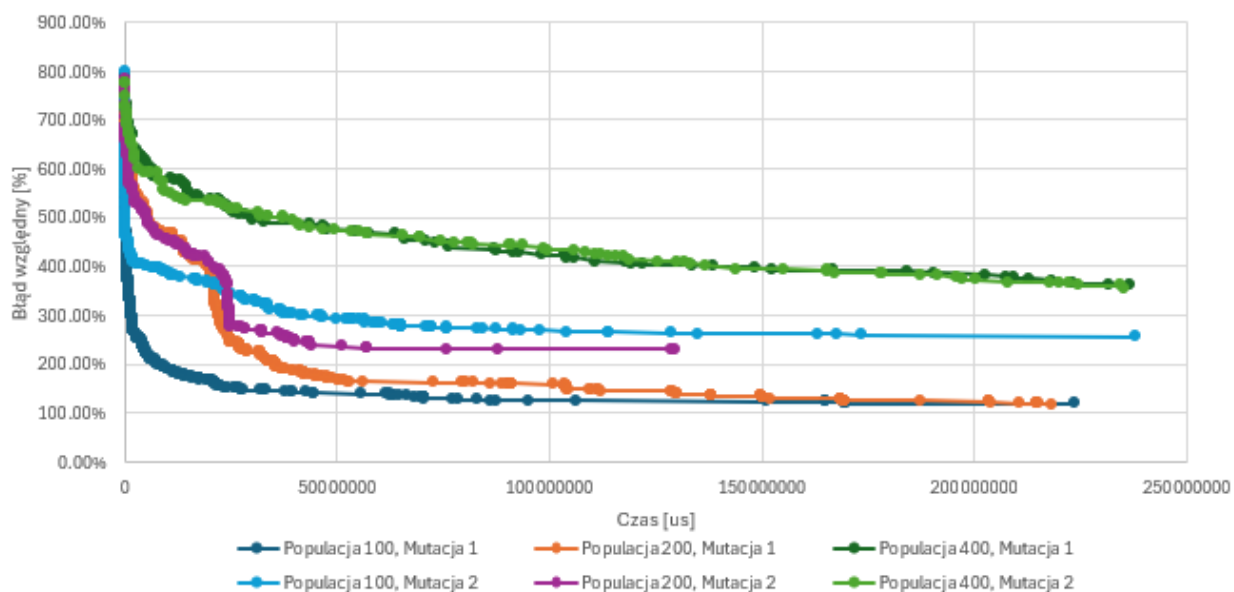
	ftv47					
Czas wykonywania	120s					
Mutacja	1			2		
Populacja	100	200	400	100	200	400
Najlepszy wynik	2640	2088	2214	2834	2407	2631
	2309	2308	3010	2647	2395	2242
	2203	2103	2197	2915	2524	2521
	2117	2263	2290	2614	2467	2396
	2390	2476	2715	2596	2370	2200
Średnia	2331.8	2247.6	2485.2	2721.2	2432.6	2398
Najlepszy wynik	2117	2088	2197	2596	2370	2200
Czas znalezienia najlepszego wyniku	6086395	62272088	45100926	12923837	98459189	85093525



4.2. Graf średni – ftv170.atsp

	ftv170					
Czas wykonywania	240s					
Mutacja	1			2		
Populacja	100	200	400	100	200	400
Najlepszy wynik	6771	6029	13622	9954	9085	13951
	6437	6217	12971	10533	9325	12546
	6062	6672	13252	9825	10150	13118
	6040	6636	13296	10051	10112	13832
	7131	6803	12741	10968	9763	13518
Średnia	6488.2	6471.4	13176.4	10266.2	9687	13393
Najlepszy wynik	6040	6029	12741	9825	9085	12546
Czas znalezienia najlepszego wyniku	223470503	218043653	236643885	237787978	129409949	235050824

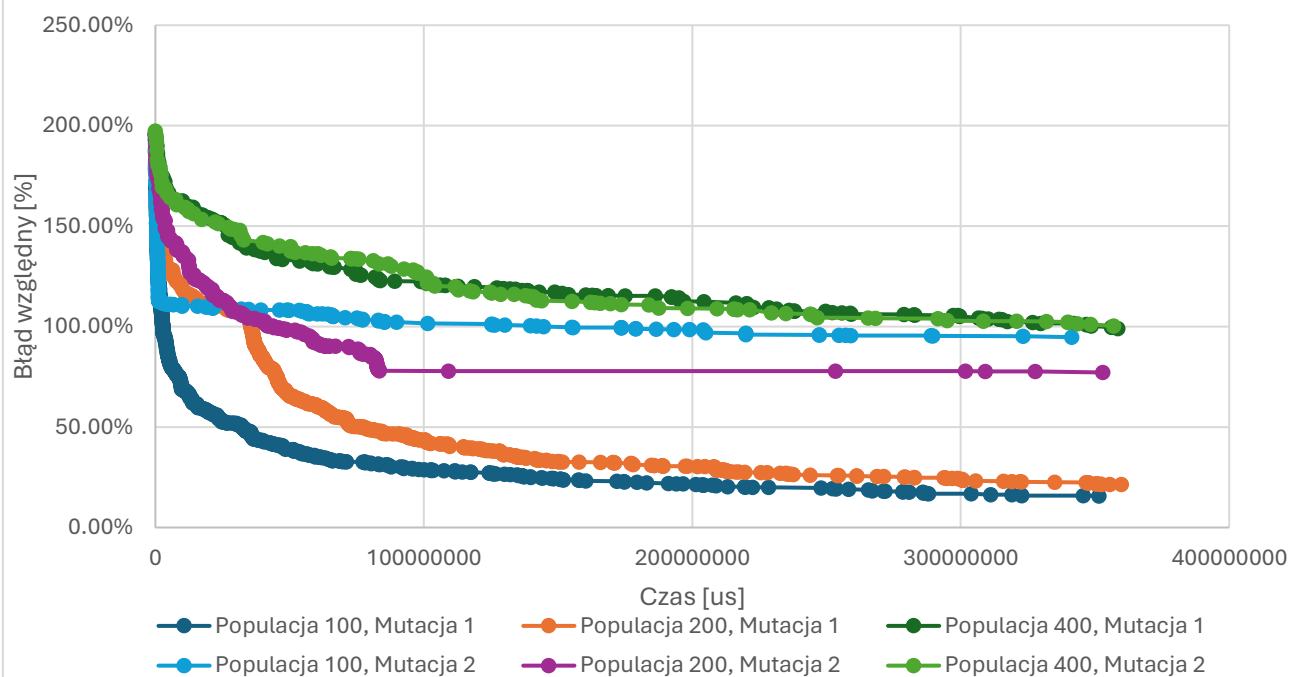
Wykres błędu względnego w czasie dla najlepszych rozwiązań dla grafu ftv170



4.3. Graf duży – rbg403.atsp

	rbg403					
Czas wykonywania	360s					
Mutacja	1			2		
Populacja	100	200	400	100	200	400
Najlepszy wynik	2865	3056	5126	5024	4367	5081
	2978	3082	5119	5010	4532	5125
	2877	2993	5080	4922	4467	4938
	2929	3023	4905	4929	4562	5012
	2854	3139	5020	4800	4446	5093
Średnia	2900.6	3058.6	5050	4937	4474.8	5049.8
Najlepszy wynik	2854	2993	4905	4800	4367	4938
Czas znalezienia najlepszego wyniku	357020627	359618498	352350129	333874503	349915348	355890642

Wykres błędu względnego w czasie dla najlepszych rozwiązań
dla grafu rbg403



5. Wnioski

Dla grafu małego najlepsze rozwiązanie otrzymaliśmy dla pierwszej metody mutacji (zamiana 2 losowych wierzchołków) oraz wielkości populacji = 200. Wyniki dla tego są do siebie zbliżone, niezależnie od wielkości populacji czy metody mutacji. Wyróżnia się jedynie najlepszy wynik oraz średnia wyników dla 2 metody mutacji i populacji = 100. Są one wyraźnie gorszy od reszty. W tym grafie zmiana wielkości populacji nie ma zbyt dużego wpływu na wyniki. Większy wpływ ma tutaj losowość algorytmu, co możemy zauważyć patrząc na czasy znalezienia najlepszych rozwiązań – rozciągają się one od ~12 sekund do ~98, przy czasie działania algorytmu = 120s. Widzimy to również w kształcie wykresu. Dowodzi to, że w tym przypadku największe znaczenie miała losowość ścieżek i ich zmian.

Dla grafu średniego najlepsze rozwiązanie ponownie otrzymaliśmy dla 1 metody mutacji i wielkości populacji = 200, jednak najlepsze rozwiązanie dla tej samej metody mutacji oraz populacji = 100 jest bardzo do niego zbliżone. Dla wielkości populacji = 400 rozwiązania są już dużo gorsze (nawet o 100%). Taki rozmiar populacji jest więc za duży dla tego grafu. Uniemożliwia on zbadanie wystarczającej ilości pokoleń (generacji) i tym samym dojście do bardziej optymalnego wyniku. Wszystkie wyniki dla 2 metody mutacji (odwrócenie fragmentu ścieżki) są od 50% do 100% gorsze. Metoda ta może zatem wprowadzać zbyt duże zmiany w osobnikach przechodzących do następnych pokoleń co znacznie utrudnia znalezienie lepszych wyników – mogą się pojawić przy odrobinie szczęścia ze względu na losowość tego algorytmu, jednak nie jest to prawdopodobne.

Dla grafu dużego możemy zaobserwować podobny trend jak dla grafu średniego: najlepsze (a zarazem zbliżone do siebie) rozwiązania znalezione zostały przy użyciu 1 metody mutacji oraz populacji = 100 lub 200. Reszta kombinacji metod mutacji i wielkości populacji daje dużo gorsze rozwiązania, tak samo najlepsze jak i średnie z wszystkich testów. Powód może być taki sam, mianowicie zbyt duża populacja zmniejsza liczbę zbadanych pokoleń w określonym czasie działania algorytmu. Być może przy zwiększeniu czasu działania algorytmu większa populacja umożliwiłaby znalezienie jeszcze lepszych rozwiązań. Dodatkowo ponownie widzimy, że 2 metoda mutacji daje gorsze wyniki.

5.1. Porównanie z Algorytmem Symulowanego Wyżarzania

Dla grafu ftv170 próbowano znaleźć jak najlepszą trasę za pomocą obydwóch algorytmów – Genetycznego oraz Symulowanego Wyżarzania. Porównywane zatem będą wyniki dla tego grafu.

Obydwa algorytmy miały ten sam czas na znalezienie rozwiązania- 4 minuty (240s). Do porównania wybrano najlepsze znalezione w testach rozwiązanie:

- Symulowane wyżarzanie – 3364
- Algorytm Genetyczny – 6029

Najlepszym wynikiem znalezionym dotychczas dla tego grafu jest 2755 co daje nam błędy następujące błędy względne:

- Symulowane wyżarzanie – 22.10%
- Algorytm Genetyczny – 118.84%

Zatem algorytm genetyczny przy takim samym czasie działania, znalazł dużo gorsze rozwiązania niż algorytm symulowanego wyżarzania. Być może możliwa jest jego optymalizacja pozwalająca na poprawę tych wyników, jednak algorytm genetyczny wydaje się dużo bardziej skomplikowany niż porównywany do niego algorytm symulowanego wyżarzania. Dalsza optymalizacja dokładałaby poziomu skomplikowania jednocześnie nie dając gwarancji lepszych wyników. Algorytm ten zatem nie nadaje się zbyt dobrze do rozwiązywania problemu TSP. Jedynym znalezionym przeze mnie jego plusem wydaje się być zauważalnie mniejsze obciążenie sprzętu, na którym wykonuje się program przy większych grafach. Jeżeli przyjrzeć się temu z tej strony, algorytm ten mógłby znaleźć zastosowanie w urządzeniach o niższej pojemności pamięci i konsumpcji energii, ewentualnie w rozwiązywaniu innych problemów. Jednak wymaga on wykonania dużej ilości obliczeń w celu otrzymania akceptowalnych wyników, więc takie urządzenie musiałoby działać przez znacznie dłuższy czas.